FINAL REPORT

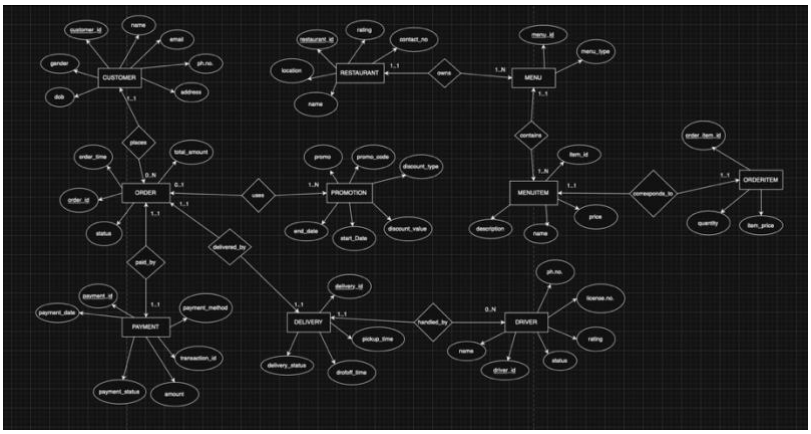
Part I: PPT presentation

Smart Food Delivery & Marketplace Management System

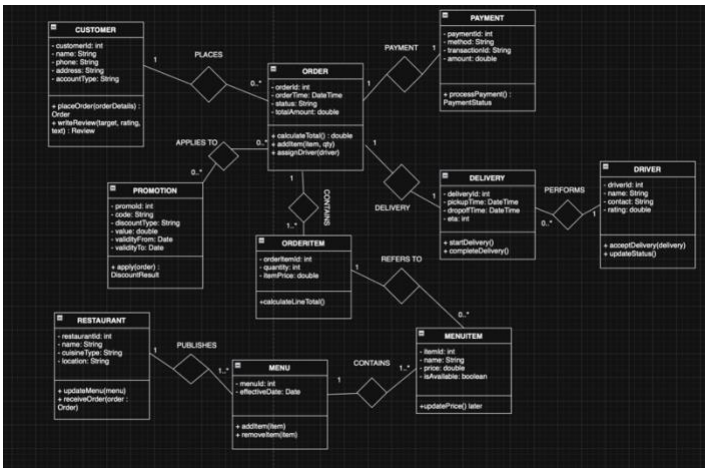
The Smart Food Delivery & Marketplace Management System is designed to model how modern food-delivery companies manage their daily operations. As cities become busier and more people rely on online platforms for meals, companies must coordinate customers, restaurants, delivery drivers, vehicles, payments, and promotions in an efficient way.

This project focuses on building a well-structured database system that supports all major activities of a food delivery platform—from browsing menus to order placement, payment processing, delivery tracking, and customer reviews. The aim is to design a system that keeps data organized, ensures smooth workflow, and supports effective decision-making.

EER MODEL



UML MODEL



Normalization Levels for Entities

CUSTOMER - Each attribute depends only on the primary key (Customer_id). No partial or transitive dependency. → 3NF.

RESTAURANT - Attributes depend entirely on Restaurant_id. No derived or repeating attributes. → 3NF.

MENU - Menu_id is the PK. Foreign key Restaurant_id creates a 1:N relation, but no transitive dependencies. → 3NF.

MENU_ITEM - Each attribute depends on Item_id. Bridge entity with no redundancy. → 3NF.

ORDER - Attributes depend only on Order_id. Foreign keys don't violate normalization. → 3NF.

ORDER_ITEM - Bridge entity resolving M:N between Order and Menu_Item. All fields depend on Order_item_id. → 3NF.

PROMOTION - Each promo identified by Promo_id. Attributes depend on it directly. → 3NF.

PAYMENT - Payment_id uniquely identifies payment details. 1:1 with Order, no redundancy. → 3NF.

DRIVER - Each attribute fully dependent on Driver_id. Atomic attributes. → 3NF.

DELIVERY - Delivery_id identifies each record uniquely. Linked 1:1 with Order, N:1 with Driver. → 3NF.

Part II: MySQL demonstration

QUERY 1 - Simple Query

```
SELECT customer_id, name, gender, account_type

FROM customers

ORDER BY customer_id

LIMIT 5;
```

customer_id	name	gender	account_type
1	Dev Patel	M	PREMIUM
2	Hitaxi Lethwala	F	STANDARD
3	Ava Brown	F	STANDARD
4	Liam Johnson	M	STANDARD
5	Noah Davis	M	STANDARD
NULL	NULL	NULL	NULL

QUERY 2 - Top Customers by Spending

```
SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_spent

FROM orders o

JOIN customers c ON o.customer_id = c.customer_id

GROUP BY c.customer_id, c.name

ORDER BY total_spent DESC

LIMIT 10;
```

customer_id	name	total_spent
1	Dev Patel	27.98
10	James Moore	24.48
6	Emma Wilson	23.98
9	Sophia Thomas	23.98
3	Ava Brown	22.48
4	Liam Johnson	18.98
2	Hitaxi Lethwala	16.48
7	Olivia Miller	13.49
5	Noah Davis	11.99
8	Mason Anderson	8.99

QUERY 3 - Most Frequently Ordered Menu Items

```
SELECT mi.item_id, mi.name AS item_name, COUNT(oi.order_item_id) AS order_count

FROM order_items oi

JOIN menu_items mi ON oi.item_id = mi.item_id

GROUP BY mi.item_id, mi.name

ORDER BY order_count DESC

LIMIT 10;
```

item_id	item_name	order_count
4	Margherita Pizza	3
2	Paneer Tikka	2
3	Veg Biryani	2
5	Pepperoni Pizza	2
8	Salmon Nigiri	2
9	Quinoa Salad	2
1	Butter Chicken	1
6	Garlic Bread	1
7	California Roll	1
10	Avocado Wrap	1

QUERY 4 - Average Order Value per Restaurant

```
SELECT r.restaurant_id,

    r.name AS restaurant_name,

    AVG(o.total_amount) AS avg_order_value

FROM orders o

JOIN restaurants r ON o.restaurant_id = r.restaurant_id

GROUP BY r.restaurant_id, r.name

ORDER BY avg_order_value DESC;
```

restaurant_id	restaurant_name	avg_order_value
1	Spice Route	21.483333
2	Pizza Plaza	21.480000
3	Sushi Sensei	17.985000
4	Green Bowl	13.985000

QUERY 5 - Deliveries Completed by Each Driver

```
SELECT d.driver_id,
    d.name AS driver_name,
    COUNT(del.delivery_id) AS deliveries_completed
FROM deliveries del
JOIN drivers d ON del.driver_id = d.driver_id
WHERE del.status = 'DELIVERED'
GROUP BY d.driver_id, d.name
ORDER BY deliveries_completed DESC;
```

driver_id	driver_name	deliveries_completed
1	Raj Mehta	3
2	Kim Lee	3
3	Carlos Ruiz	2
4	Mina Park	2

QUERY 6 - Detailed Breakdown of Each Order

```
SELECT o.order_id,
    c.name AS customer,
    r.name AS restaurant,
    mi.name AS item_name,
    oi.quantity,
    oi.item_price
FROM order_items oi
JOIN orders o      ON oi.order_id = o.order_id
JOIN customers c   ON o.customer_id = c.customer_id
JOIN restaurants r  ON o.restaurant_id = r.restaurant_id
JOIN menu_items mi  ON oi.item_id = mi.item_id
ORDER BY o.order_id, mi.name;
```

order_id	customer	restaurant	item_name	quantity	item_price
1	Dev Patel	Spice Route	Paneer Tikka	1	12.49
2	Hitaxi Lethwala	Pizza Plaza	Garlic Bread	1	5.49
2	Hitaxi Lethwala	Pizza Plaza	Margherita Pizza	1	10.99
3	Ava Brown	Sushi Sensei	California Roll	1	9.99
3	Ava Brown	Sushi Sensei	Salmon Nigiri	1	13.49
4	Liam Johnson	Green Bowl	Avocado Wrap	1	9.99
4	Liam Johnson	Green Bowl	Quinoa Salad	1	8.99
5	Noah Davis	Spice Route	Veg Biryani	1	11.99
6	Emma Wilson	Pizza Plaza	Margherita Pizza	1	10.99
6	Emma Wilson	Pizza Plaza	Pepperoni Pizza	1	12.99
7	Olivia Miller	Sushi Sensei	Salmon Nigiri	1	13.49
8	Mason Anderson	Green Bowl	Quinoa Salad	1	8.99

QUERY 7 - Payment Method Usage Summary

```
SELECT method AS payment_method,
    COUNT(payment_id) AS number_of_payments,
    SUM(amount) AS total_amount_processed
FROM payments
GROUP BY method
ORDER BY total_amount_processed DESC;
```

payment_method	number_of_payments	total_amount_processed
CARD	7	149.37
WALLET	2	31.47
CASH_ON_DELIVERY	1	11.99

QUERY 8 - Restaurant(s) With Highest Average Order Value

```
SELECT
   r.restaurant_id,
   r.name AS restaurant_name,
   AVG(o.total_amount) AS avg_order_value
FROM orders o
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING AVG(o.total_amount) >= ALL (
   SELECT AVG(o2.total_amount)
   FROM orders o2
   GROUP BY o2.restaurant_id
);
```

restaurant_id	restaurant_name	avg_order_value
1	Spice Route	21.483333

QUERY 9 - Customers Who Used Promotions

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
   SELECT 1
   FROM orders o
   WHERE o.customer_id = c.customer_id
     AND o.promo_id IS NOT NULL
);
```

customer_id	name
1	Dev Patel
3	Ava Brown
6	Emma Wilson
9	Sophia Thomas
NULL	NULL

QUERY 10 - All People on Platform (UNION)

```
SELECT name, 'CUSTOMER' AS role
FROM customers
UNION
SELECT name, 'DRIVER' AS role
FROM drivers;
```

name	role
Dev Patel	CUSTOMER
Hitaxi Lethwala	CUSTOMER
Ava Brown	CUSTOMER
Liam Johnson	CUSTOMER
Noah Davis	CUSTOMER
Emma Wilson	CUSTOMER
Olivia Miller	CUSTOMER
Mason Anderson	CUSTOMER
Sophia Thomas	CUSTOMER
James Moore	CUSTOMER
Raj Mehta	DRIVER
Kim Lee	DRIVER
Carlos Ruiz	DRIVER
Mina Park	DRIVER

QUERY 11 - Daily Order Stats + Percentage

```
SELECT
  d.order_date,
  d.total_orders,
  d.total_revenue,
  ROUND(
    100.0 * d.total_orders / (SELECT COUNT(*) FROM orders),
    2
  ) AS pct_of_all_orders
FROM (
  SELECT
    DATE(order_time) AS order_date,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS total_revenue
  FROM orders
  GROUP BY DATE(order_time)
) AS d
ORDER BY d.order_date;
```

order_date	total_orders	total_revenue	pct_of_all_orders
2025-11-09	10	192.83	100.00

Part III: NoSQL demonstration

QUERY - 1 SIMPLE QUERY

```
[
 { $match: { account_type: "PREMIUM" } }
]
```

```
customer_id : 1
name : "Dev Patel"
gender : "M"
date_of_birth : "1999-08-10"
phone_number : "+1-617-000-1000"
address : "Boston, MA"
account_type : "PREMIUM"
_id: ObjectId('692caa5e56be661ab227e8e6')
```

QUERY - 2 COMPLEX QUERY

```
[
 {
  $match: {
   status: "DELIVERED",
   total_amount: { $gt: 18 },
   "payment.method": "CARD"
  }
 }
```

```
_id: ObjectId('692cab6156be661ab227e8fb')
promo_code : "WELCOME10"
order_time : "2025-11-09 18:10:00"
total_amount : 27.98
status : "DELIVERED"
delivery : Object
order_id : 1
customer_id : 1
restaurant_id : 1
items : Array (2)
payment : Object
```

```
total_amount : 18.98
status : "DELIVERED"
items : Array (2)
delivery : Object
payment : Object
_id: ObjectId('692cab6156be661ab227e8fe')
order_id : 4
customer_id : 4
restaurant_id : 4
promo_code : null
order_time : "2025-11-09 18:25:00"
```

QUERY - 3 AGGREGATE QUERY

```
[
 { $match: { status: "DELIVERED" } },
 {
  $group: {
   _id: "$customer_id",
   total_spent: { $sum: "$total_amount" }
  }
 },
 {
  $lookup: {
   from: "customers",
   localField: "_id",
   foreignField: "customer_id",
   as: "customer_info"
  }
 },
 { $unwind: "$customer_info" }
 {
  $project: {
   _id: 0,
   customer_id: "$_id",
   customer_name: "$customer_info.name",
   total_spent: 1
  }
 },
 { $sort: { total_spent: -1 } },
 { $limit: 5 }
]
```

```
total_spent : 27.98
customer_id : 1
customer_name : "Dev Patel"
```

```
total_spent : 22.48
customer_id : 3
customer_name : "Ava Brown"
```

```
total_spent : 18.98
customer_id : 4
customer_name : "Liam Johnson"
```

```
total_spent : 16.48
customer_id : 2
customer_name : "Hitaxi Lethwala"
```

```
total_spent : 11.99
customer_id : 5
customer_name : "Noah Davis"
```

Part IV: Application demonstration

db_config.py — Database Connection Settings

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="12345678",
        database="smart_food"
    )
```

queries.py — SQL Query Functions

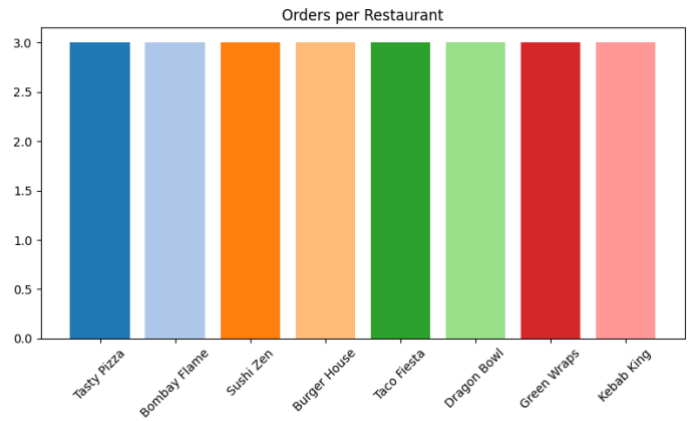
```
from db_config import get_connection
import pandas as pd

def run_query(sql):
    """
    Runs SQL and returns a pandas DataFrame
    """
    conn = get_connection()
    try:
        df = pd.read_sql(sql, conn)
    finally:
        conn.close()
    return df


# QUERY 1 — Orders per Restaurant
def q1_orders_per_restaurant():
    return run_query("""
        SELECT r.name AS restaurant, COUNT(o.order_id) AS total_orders
        FROM orders o
        JOIN restaurants r ON o.restaurant_id = r.restaurant_id
        GROUP BY r.name;
    """)
# QUERY 2 — Top Selling Menu Items
def q2_top_selling_items():
    return run_query("""
        SELECT m.name, SUM(oi.quantity) AS qty
        FROM menu_item m
        JOIN order_item oi ON m.item_id = oi.item_id
```

```
        GROUP BY m.name
        ORDER BY qty DESC
        LIMIT 10;
    """)
# QUERY 3 — Revenue per Restaurant
def q3_revenue_per_restaurant():
    return run_query("""
        SELECT r.name AS restaurant, SUM(o.total_amount) AS revenue
        FROM orders o
        JOIN restaurants r ON o.restaurant_id = r.restaurant_id
        GROUP BY r.name;
    """)
# QUERY 4 — Daily Order Trend
def q4_daily_orders():
    return run_query("""
        SELECT DATE(order_time) AS day, COUNT(order_id) AS total_orders
        FROM orders
        GROUP BY DATE(order_time)
        ORDER BY day;
    """)
# QUERY 5 — Payment Method Distribution
def q5_payment_methods():
    return run_query("""
        SELECT payment_method, COUNT(*) AS usage_count
        FROM orders
        GROUP BY payment_method;
    """)
# QUERY 6 — Restaurant Ratings
def q6_restaurant_ratings():
    return run_query("""
        SELECT name AS restaurant, rating
        FROM restaurants;
    """)
```

app.py — Main Application & Chart Generator

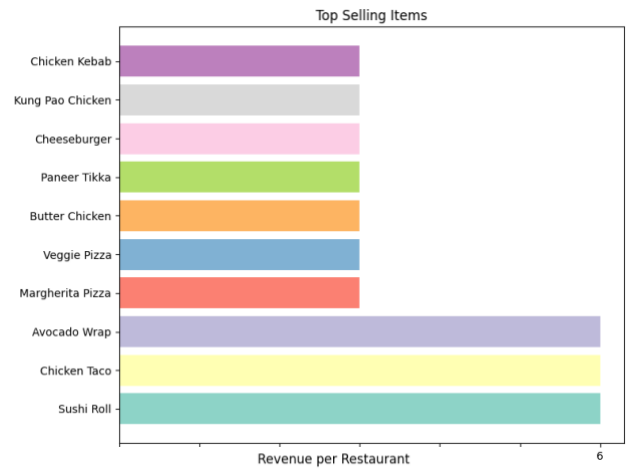
Query 1 — Orders per Restaurant

```
df1 = q1_orders_per_restaurant()
plt.figure(figsize=(8,5))
plt.bar(df1['restaurant'], df1['total_orders'], color=colors2)
plt.title("Orders per Restaurant")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("charts/01_orders_per_restaurant.png")
plt.close()
print("✓ Saved Chart 1")
```



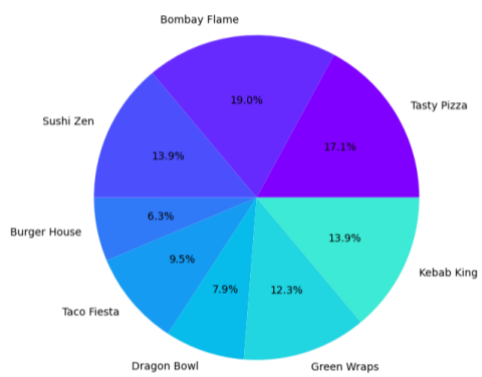
Query 2 — Top Selling Menu Items

```
df2 = q2_top_selling_items()
plt.figure(figsize=(8,6))
plt.barh(df2['name'], df2['qty'], color=colors1)
plt.title("Top Selling Items")
plt.tight_layout()
plt.savefig("charts/02_top_selling_items.png")
plt.close()
print("✓ Saved Chart 2")
```



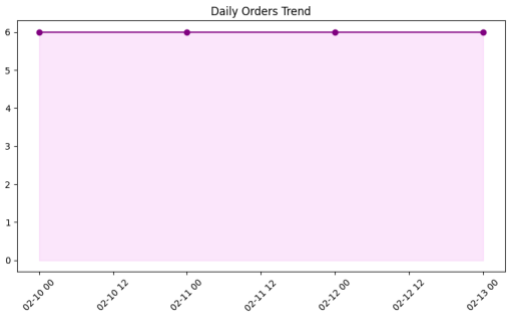
Query 3 — Revenue per Restaurant

```
df3 = q3_revenue_per_restaurant()
plt.figure(figsize=(7,7))
plt.pie(
    df3['revenue'],
    labels=df3['restaurant'],
    autopct='%1.1f%%',
    colors=colors3
)
plt.title("Revenue per Restaurant")
plt.savefig("charts/03_revenue_distribution.png")
plt.close()
print("✓ Saved Chart 3")
```



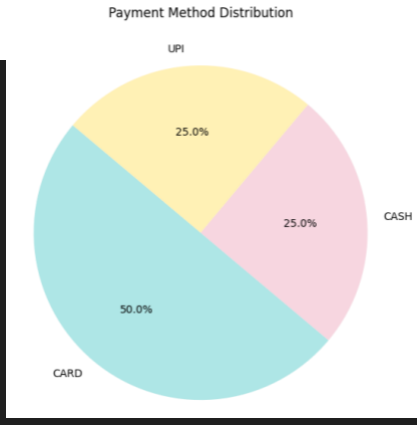
Query 4 — Daily Orders Trend

```
df4 = q4_daily_orders()
plt.figure(figsize=(8,5))
plt.plot(df4['day'], df4['total_orders'], marker='o', color='purple')
plt.fill_between(df4['day'], df4['total_orders'], alpha=0.2, color='violet')
plt.xticks(rotation=45)
plt.title("Daily Orders Trend")
plt.tight_layout()
plt.savefig("charts/04_daily_orders_trend.png")
plt.close()
print("✓ Saved Chart 4")
```



Query 5 — Payment Method Usage

```
df5 = q5_payment_methods()
count_col = [c for c in df5.columns if df5[c].dtype != object][0]
method_col = [c for c in df5.columns if df5[c].dtype == object][0]
pastel_colors = ["#AEE6E6", "#F7D6E0", "#FFF1B5"]
plt.figure(figsize=(7,6))
plt.pie(
    df5[count_col],
    labels=df5[method_col],
    autopct='%1.1f%%',
    colors=pastel_colors,
    startangle=140
)
plt.title("Payment Method Distribution")
plt.tight_layout()
plt.savefig("charts/05_payment_methods_pastel.png")
plt.close()
print("✓ Saved Chart 5")
```



Query 6 — Restaurant Ratings Comparison

```
df6 = q6_restaurant_ratings()
x_col = [col for col in df6.columns if col != 'rating'][0]
```

```
num_points = len(df6)
point_colors = plt.cm.cool(np.linspace(0, 1, num_points))
plt.figure(figsize=(8,6))
plt.scatter(df6[x_col], df6['rating'], s=180, c=point_colors, edgecolors='black')
plt.xticks(rotation=45)
plt.title("Restaurant Rating Comparison", fontsize=14)
plt.tight_layout()
plt.savefig("charts/06_ratings_scatter.png")
plt.close()
print("✓ Saved Chart 6")
```

Summary

This project presents the end-to-end design and implementation of a Smart Food Delivery & Marketplace Management System**,** modeled on how modern delivery platforms manage customers, restaurants, orders, payments, and delivery operations. The goal was to create a robust data management solution capable of supporting day-to-day transactions while enabling advanced analytics for business decision-making.The system was first conceptualized using EER and UML models**,** capturing key entities—such as Customer, Restaurant, Menu, Order, Driver, Payment, and Delivery—and defining their relationships and operational behaviors. All entities were normalized to Third Normal Form (3NF) to ensure data consistency, eliminate redundancy, and support scalable database operations.A fully functional MySQL relational database was implemented, followed by the execution of a diverse set of SQL queries ranging from simple selections to aggregate analytics, multi-table joins, nested subqueries, correlated subqueries, and set operations. These queries answered critical business questions, including customer spending, item popularity, restaurant performance, delivery efficiency, and payment behavior.

To demonstrate flexibility beyond relational storage, **a** NoSQL model was developed in MongoDB using embedded documents for orders, payments, and deliveries. Multiple MongoDB queries—from simple filters to complex aggregation pipelines—highlighted how document-based databases can simplify certain analytical operations.Finally, a Python application was created to connect to the MySQL database, execute SQL queries, and generate visual analytics using Matplotlib. The resulting charts—including order trends, revenue distribution, top-selling items, payment usage, and restaurant ratings—illustrate how the system enables data-driven insights for operational improvement.Overall, the project delivers a comprehensive data management solution integrating conceptual modeling, relational and non-relational databases, query-driven analytics, and Python-based visualization. It demonstrates how a well-designed backend architecture can support both transactional workloads and strategic analysis in a modern food delivery environment.